



MODUL 14 · SEO, METADATA & PERFORMANCE

Metadata API & generateMetadata

Static dan dynamic metadata. Open Graph, Twitter Cards, icons. Semua native di Next.js.



Static Metadata — export const metadata

Untuk halaman yang tidak bergantung data dinamis. Cukup export satu object.

```
import type { Metadata } from "next"

export const metadata: Metadata = {
  // metadataBase: WAJIB untuk resolve OG image URL!
  metadataBase: new URL(process.env.NEXT_PUBLIC_APP_URL!),
  // Gunakan new URL() bukan string biasa!

  // Title template: "%s | Brand" untuk child pages
  title: {
    template: "%s | EasyCoding",
    default: "EasyCoding — Kursus Next.js Terbaik",
  },
  description: "Belajar Next.js, Prisma & PostgreSQL dari dasar hingga deploy.",
  // Open Graph
  openGraph: {
    type: "website",
    siteName: "EasyCoding",
    title: "EasyCoding — Kursus Next.js Terbaik",
    description: "Belajar Next.js, Prisma & PostgreSQL.",
    images: [{
      url: "/og-default.jpg", // relatif → diresolution dengan metadataBase
      width: 1200,
      height: 630,
      alt: "EasyCoding",
    }],
  },

  // Twitter / X Card
  twitter: {
    card: "summary_large_image",
    title: "EasyCoding",
    description: "Kursus Next.js Terbaik",
    images: ["/og-default.jpg"],
  },
}
```

metadataBase Wajib!

Tanpa metadataBase:
OG images dengan URL relatif
tidak bisa di-resolve oleh
crawler (Facebook, Twitter, dll)

✓ Benar:
metadataBase: new URL(
 process.env.NEXT_PUBLIC_APP_URL!
)

✗ Salah:
metadataBase: 'https://...'
(string, bukan URL object)

Child pages set title singkat:
metadata = { title: "Blog" }
→ Render: "Blog | EasyCoding"
(dari template root layout)

SEO & PERFORMANCE

generateMetadata — Metadata Dinamis

Untuk halaman dinamis (post detail, user profile). Fetch data untuk metadata.

```
import type { Metadata } from 'next'
import { notFound } from 'next/navigation'
import { cache } from 'react'

// React.cache(): deduplication untuk Prisma queries
// (fetch() di-dedup otomatis, Prisma tidak — perlu cache() manual)
const getPost = cache(async (slug: string) => {
  return db.post.findUnique({
    where: { slug },
    select: {
      title: true,
      excerpt: true,
      coverImage: true,
      author: { select: { name: true } },
    },
  })
})

// generateMetadata dipanggil sebelum render page
export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>,
}): Promise<Metadata> {
  const { slug } = await params
  const post = await getPost(slug) // ← getPost juga dipanggil di page() bawah
  if (!post) return { title: 'Post Not Found' }

  const ogImage = post.coverImage ?? '/og-default.jpg'
  return {
    title: post.title, // → "{title} | EasyCoding"
    description: post.excerpt ?? undefined,
    openGraph: {
      type: 'article',
      title: post.title,
      images: [{ url: ogImage, width: 1200, height: 630 }],
    },
    twitter: {
      card: 'summary_large_image',
      title: post.title,
      images: [ogImage],
    },
    alternates: { canonical: `/blog/${slug}` }, // URL kanonik
  }
}

export default async function PostPage({ params }) {
  const { slug } = await params
  const post = await getPost(slug) // ← Prisma query TIDAK di-duplikasi! cache()
  if (!post) notFound()
  return <article>...</article>
}
```

Icons & OG Images — File-based Convention

Taruh file di folder yang tepat → Next.js auto-generate meta tags. Tidak perlu konfigurasi.

Struktur file untuk icons & OG

```

app/
├── favicon.ico           # Tab icon browser
├── icon.png              # App icon (PWA)
├── icon.svg              # Scalable icon
├── apple-icon.png        # iOS home screen
├── opengraph-image.jpg   # Default OG image
├── twitter-image.jpg     # Twitter card image
├── blog/
│   └── [slug]/
│       └── opengraph-image.tsx # Dynamic OG per post!
└── layout.tsx
  
```

```

# Next.js otomatis generate:
# <link rel="icon" href="/favicon.ico">
# <link rel="apple-touch-icon" href="/apple-icon.png">
# <meta property="og:image" content="/opengraph-
image.jpg">
# dll.
  
```

```

# Ukuran yang direkomendasikan:
# favicon.ico:      32×32 atau 64×64
# opengraph-image:  1200×630
# apple-icon:       180×180
  
```

opengraph-image.tsx — dynamic OG via JSX

```

// app/blog/[slug]/opengraph-image.tsx
// Generate OG image per post dengan JSX + CSS!
import { ImageResponse } from "next/og"
import { getPost } from "@lib/data/posts"

export const runtime = "edge"
export const size = { width: 1200, height: 630 }

export default async function OgImage({
  params,
}: { params: Promise<{ slug: string }> }) {
  const { slug } = await params
  const post = await getPost(slug)

  return new ImageResponse(
    <div style={{
      background: "#0C1220",
      width: "100%", height: "100%",
      display: "flex", flexDirection: "column",
      justifyContent: "center", padding: 80,
    }}>
      <p style={{ color: "#1E86CF", fontSize: 28 }}>
        EasyCoding Blog
      </p>
      <h1 style={{ color: "white", fontSize: 60, margin: 0 }}>
        {post?.title ?? "Post"}
      </h1>
    </div>
  )
}
  
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa metadataBase wajib di Next.js App Router?

- A) Tanpa ini metadata tidak digenerate
- B) Agar URL relatif (OG images, canonical) bisa di-resolve menjadi URL absolut oleh crawler
- C) Hanya untuk deployment di Vercel

Q2 — Jika generateMetadata dan page() keduanya call getPost(slug) dengan Prisma, berapa kali query DB dijalankan?

- A) Dua kali — setiap call terpisah
- B) Satu kali — jika getPost di-wrap dengan React.cache()
- C) Tergantung apakah pakai 'use cache'

Q3 — Apa fungsi alternates.canonical di metadata?

- A) Menentukan URL untuk sitemap
- B) Mendefinisikan URL kanonik — mencegah duplicate content di search engine
- C) Alias untuk redirect

Tugas Mandiri

Implementasikan metadata lengkap untuk project blog.

01 Root Metadata

Di `app/layout.tsx`: set `metadataBase` (new URL!), title template, description, openGraph, twitter. Tambahkan `NEXT_PUBLIC_APP_URL` ke `.env`.

02 Post Metadata

Buat `generateMetadata` di `app/blog/[slug]/page.tsx` dengan `React.cache()`. Fetch post → title, description, OG image, canonical URL.

03 Icons

Taruh `favicon.ico` di `app/`. Buat `opengraph-image.jpg` (1200×630) di `app/`. Test: buka DevTools → Elements → cek meta tags yang digenerate.

04 Dynamic OG

Buat `app/blog/[slug]/opengraph-image.tsx` dengan `ImageResponse`. Test di `opengraph.xyz` dengan URL blog post kamu.



Test OG: `opengraph.xyz` atau `metatags.io` — masukkan URL dan lihat preview OG image serta meta tags.

Yang Sudah Kita Pelajari

- ✓ metadataBase: new URL(APP_URL) wajib — resolve OG image URL relatif ke absolut.
- ✓ Title template di root layout: '%s | Brand'. Child pages cukup set title singkat.
- ✓ generateMetadata: fetch data untuk metadata dinamis. Wrap Prisma dengan React.cache().
- ✓ File-based icons: favicon.ico, apple-icon.png, opengraph-image.jpg di app/ folder.
- ✓ opengraph-image.tsx: dynamic OG image dengan JSX + ImageResponse. Edge runtime.

→ Selanjutnya: **Bab 2 — sitemap.ts & robots.ts**

